

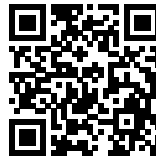
ComEng - Ergänzung

FS 2026 – Prof. Lars Kamm

Autoren:
Mirko Ratti

Version:
1.0.20260706

<https://github.com/mirkoratti/FS2026>



Inhaltsverzeichnis

7	Instruction Set Architecture (ISA)	11			
8	Logica della ALU e Program Status Register	11			
9	Componenti del Sistema (SCS)	11			
10	Stack e Gestione Memoria	11			
11	Confronto Architetture ARM Cortex	11			
11.1	Evoluzione Cortex-M0 vs M0+	11			
12	Gestione degli Interrupt (NVIC)	11			
12.1	NVIC Special Features	11			
12.2	Registri di Mascheramento	11			
12.3	Latenza e Stacking	11			
12.4	Preemption vs Subpriority	11			
			12.5	Soluzioni alternative & Segnali IRQ	11
			13	Ottimizzazione e Strutture Dati	11
			14	Adressierungsarten ARM (Cortex-M0)	12
			14.1	Register Direkt, Immediate	12
			14.2	Register Indirekt (ptr)	12
			14.3	PC-Relative & Literal Pool	12
			14.4	Registerlisten & Stack (SP-Relative)	12
			14.5	Stack Framing	12
			15	Il Processo di Compilazione	12
			15.1	Analisi del Compilatore	12
			16	Esempi	12

Hex	A	B	C	D	E	F
Dec	10	11	12	13	14	15
Bin	1010	1011	1100	1101	1110	1111

7 Instruction Set Architecture (ISA)

- **ARM (32-bit) (Obsoleto):** 32 bit, massime prestazioni ma occupano maggior spazio in memoria.
- **Thumb (16-bit):** sottoinsieme 16 bit, Ideale per ridurre il footprint della memoria (circa 30% di risparmio)
- **Thumb-2 (16/32-bit):** lunghezza variabile (16 e 32 bit). Permette di ottenere una densità di codice simile a Thumb con prestazioni analoghe al set ARM a 32 bit.
- **RISC:** Istruzioni semplici e veloci eseguite in un ciclo di clock; usa un'architettura Load/Store (calcoli solo nei registri) e richiede più righe di codice Assembly.
- **CISC:** Istruzioni complesse e potenti che compiono più operazioni insieme; permette di eseguire calcoli direttamente sulla memoria RAM, riducendo le righe di codice Assembly.

8 Logica della ALU e Program Status Register

- **N (Negative):** Se $MSB = 1$, la flag è impostata a 1.
- **Z (Zero):** tutti i bit del risultato sono pari a zero.
- **C (Carry):** Indica un riporto oltre il bit più significativo.
 - Fondamentale per operazioni *unsigned*. **Invertito per la sottrazione!**
 - **Meccanismo sottrazione:** $(A + NOT(B) + 1)$. Zweierkompl.
 - * $C = 1 \implies A \geq B$ (Nessun prestito / Not-Borrow).
 - * $C = 0 \implies A < B$ (Prestito avvenuto / Borrow).
- **V (oVerflow):** Indica un errore di segno nel calcolo in complemento a due, Fondamentale per operazioni *signed*.
 - $(+) + (+) = (-)$ Impossibile
 - $(-) + (-) = (+)$ Impossibile

8.0.1 Interpretazione del Segno e Salti Condizionati

Poiché la ALU aggiorna simultaneamente sia la flag *C* che la flag *V*, la distinzione tra variabile `int` e `uint` non risiede nell'hardware, ma nel software.

9 Componenti del Sistema (SCS)

- **HardFaults:** Sempre attivati; gestiscono errori di Bus, Usage e Memory Management. Permettono una reazione rapida agli errori critici.
- **Wake-up Interrupt:** Consente la disattivazione gerarchica dei componenti per ridurre il consumo energetico (modalità Sleep/Stop).
- **Memory Protection Unit (MPU):** Genera eccezioni in caso di accessi non autorizzati alla memoria.
- **Debug Unit:** Fornisce fino a 4 breakpoint hardware per il debugging del codice. Core Debug Control, Breakpoint Unit (BP), Data Watchpoint Unit (DWT), ROM Table
- Un contatore a **24-bit** che fornisce una base dei tempi costante per il sistema operativo o i cicli di programma.

10 Stack e Gestione Memoria

- **Direzione:** Cresce verso indirizzi di memoria inferiori.
- **Stack Pointer (SP):** Punta sempre al "top of stack". vietato leggere o scrivere al di sotto di SP.
- **Allineamento:** allocato in step di 8 byte.
- **Velocità:** due volte più rapido del RAM/ROM pointer, SP è già disponibile.

11 Confronto Architetture ARM Cortex

Cortex-A (Application): Calcolo generico ad alte prestazioni (Linux/Android).

Cortex-R (Real-Time): Bassa latenza e alta affidabilità (automotive/medico).

Cortex-M (Microcontroller): Ridotto consumo energetico, interrupt deterministici.

Core	Thumb	Thumb-2	HW Mult.	HW Div.	Math Sat.	DSP	FPU	Arch
Cortex-M0	Mostly	Subset	1/32 cyc	No	No	No	No	ARMv6-M
Cortex-M0+	Mostly	Subset	1/32 cyc	No	No	No	No	ARMv6-M
Cortex-M1	Mostly	Subset	3/33 cyc	No	No	No	No	ARMv6-M
Cortex-M3	Fully	Fully	1 cyc	Yes	Yes	No	No	ARMv7-M
Cortex-M4/M7	Fully	Fully	1 cyc	Yes	Yes	Yes	Optional	ARMv7E-M

11.1 Evoluzione Cortex-M0 vs M0+

- **M0+ Pipeline:** Ridotta a 2 stadi per maggiore efficienza.
 - + Es wird in jedem Cycle ein Befehl ausgeführt
- **Accesso I/O:** Supporto per Single-Cycle GPIO.
 - Bei sprüngen muss die Pipeline 'geleert' werden
- **Debugging:** Micro Trace Buffer (MTB) opzionale per il tracciamento dei salti.
- **MPU**

12 Gestione degli Interrupt (NVIC)

- Il **Nested Vector Interrupt Controller** gestisce la prioritizzazione e il passaggio tra il programma di background e le **Interrupt Service Routine (ISR)**.
- **L'LSB in ogni vettore indica la modalità di salto (1: thumb, 0: ARM) nei Cortex-M sempre 1** (Anche indirizzi in LR con B. .)
- Solitamente la VT (vector table) viene salvata in memoria fra il boot code e la seconda istruzione.
- Tramite VTOR (Vector Table Offset Register) è possibile cambiare la posizione della tabella in memoria.
- **Stati dell'Interrupt:** *Pending* (richiesto dalla periferica, azzerato automaticamente dall'hardware all'ingresso della ISR), *Active* (in esecuzione in Handler Mode), *Inattive*.

12.1 NVIC Special Features

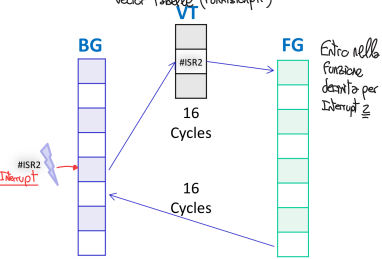
- **Tail-Chaining:** IRQ a bassa priorità è pendente al termine di ISR attiva, la CPU salta l'Unstacking e lo Stacking. Fa solo un nuovo Vector Fetch (da 15 → transizione di soli **6 cicli**).
- **Late Arrival:** IRQ ad alta priorità si attiva durante lo stacking di un IRQ a bassa priorità, la CPU completa stacking ma esegue direttamente Vector Fetch dell'IRQ più urgente.
- **Pop-Preemption:** IRQ arriva durante l'Unstacking (Pop) finale, l'hardware abbandona il Pop e devia subito alla nuova ISR in soli **6 cicli**.

12.2 Registri di Mascheramento

- **PRIMASK:** 1 bit. Disabilita tutti gli interrupt programmabili (tranne HardFault e NMI).
- **FAULTMASK:** 1 bit. Disabilita interrupt e tutte le eccezioni (tranne NMI).
- **BASEPRI:** Disabilita interrupt con priorità numerica maggiore o uguale (urgenza inferiore) a un valore impostato.

12.3 Latenza e Stacking

- **Interrupt Latency:** Tempo fisso d'ingresso nella ISR senza overhead software. **Cortex-M0+:** 15–16 cicli di clock. **Cortex-M3/M4:** 12 cicli.
- Le funzioni ISR sono di tipo `void ISR_Handler(void)`: non accettano parametri e non restituiscono valori.

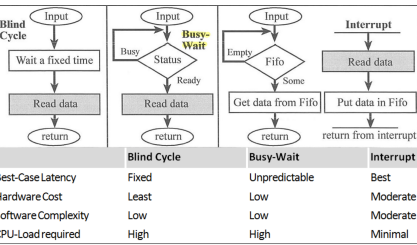


12.4 Preemption vs Subpriority

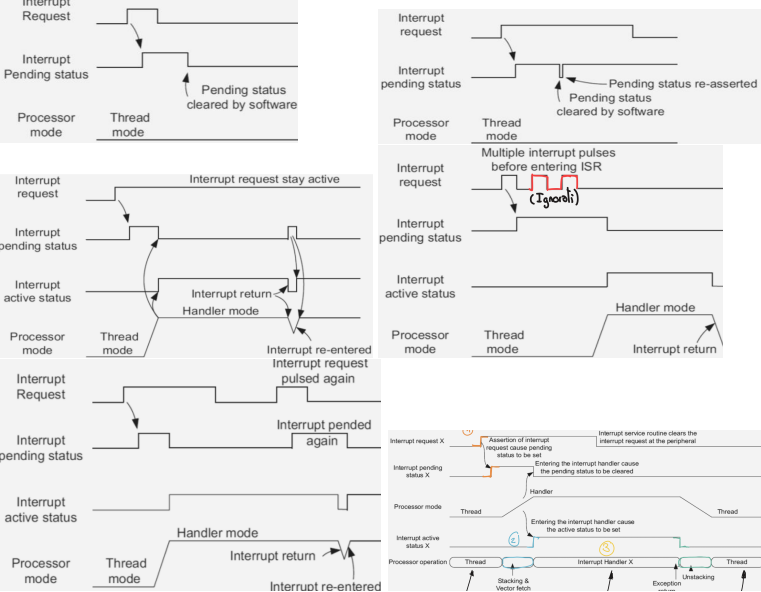
- **Cortex-M0+:** Supporta solo 2 bit di priorità utilizzabili solo come Preemption.
- **Cortex-M3/M4:** Tramite il *Priority Grouping*, gli 8 bit di priorità si dividono in:
 - **Preemption bits:** Decidono se un interrupt può interrompere fisicamente una ISR in esecuzione (*nesting*).
 - **Subpriority bits:** Criterio di spargio temporale se due IRQ arrivano insieme

Exception Number	Vectors	CMSIS Interrupt Number	Memory Address
47	Interrupt#31 Vector	31	0x0000'00BC
...	...	...	...
18	Interrupt#2 Vector	2	0x0000'0048
17	Interrupt#1 Vector	1	0x0000'0044
16	Interrupt#0 Vector	0	0x0000'0040
15	SysTick Vector	-1	0x0000'003C
14	PendSV Vector	-2	0x0000'0038
13	Not Used	NA	0x0000'0034
12	Not Used	NA	0x0000'0030
11	SVC Vector	-5	0x0000'002C
10	Not Used	NA	0x0000'0028
9	Not Used	NA	0x0000'0024
8	Not Used	NA	0x0000'0020
7	Not Used	NA	0x0000'001C
6	Not Used	NA	0x0000'0018
5	Not Used	NA	0x0000'0014
4	Not Used	NA	0x0000'0010
3	Hard Fault Vector	-13	0x0000'000C
2	NMI Vector	-14	0x0000'0008
1	Reset Vector	NA	0x0000'0004
0	MSP Initial Value	NA	0x0000'0000

12.5 Soluzioni alternative & Segnali IRQ



1. Software Clearing
2. Software Clearing mit Reset
3. ISR Reentry (Pulsed IRQ durante ISR)
4. Repulsed IRQ
5. Pulsed IRQ (Level stay active)



13 Ottimizzazione e Strutture Dati

13.0.1 Switch Case

- **Pro:** Numero di cicli di clock definito e prevedibile.
- **Contro:** La condizione deve essere necessariamente una costante di tipo `int`.
- **Nota:** L'ordine dei case è rilevante per l'efficienza della ricerca.

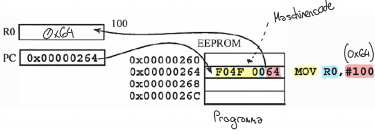
13.0.2 Efficienza delle Strutture Dati

- **Array Locali:** Molto **inefficienti**. alto numero di cicli solo per l’inizializzazione nello stack frame
- **Strutture:** È consigliabile **definire le strutture globalmente**. L’inizializzazione locale su stack comporta un overhead elevato di cicli di clock.
- **Bitfields:** Estremamente **inefficienti**. Sebbene risparmino memoria, richiedono complesse operazioni di *Read-Modify-Write* (LDR, Shift, Mask, OR, STR)

14 Adressierungsarten ARM (Cortex-M0)

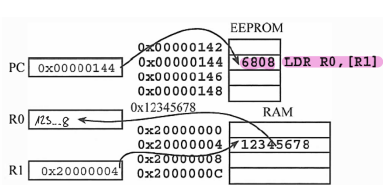
14.1 Register Direkt, Immedia-

- **Immediate:** MOV R0, #100 → Valore caricato direttamente.
- **Register:** MOVs R2, R1 → Copia valore tra registri.
- **Limitazioni:** Gli immediati sono codificati nell’istruzione (es. 8-bit per MOVs).



14.2 Register Indirekt (ptr)

- **Semplice:** LDR R0, [R1] → R0 = valore all’indirizzo R1.
- **Con Offset:** LDR R0, [R1, #4] → R0 = valore all’indirizzo R1 + 4 (utile per array/struct).
- **Con Index:** LDR R0, [R1, R2] → R0 = valore all’indirizzo R1 + R2.



14.3 PC-Relative & Literal Pool

- **Problema:** Istruzioni da 32-bit non possono contenere costanti da 32-bit.
- **Soluzione:** L’assembler salva la costante nel **Literal Pool** (zona memoria vicina).
- **Accesso:** LDR R0, =0x12345678 viene tradotto in LDR R0, [PC, #offset].
- Richiedono sempre **due accessi** alla memoria

14.4 Registerlisten & Stack (SP-Relative)

- **Multiple:** LDM R7!, {R0-R3, R5} → Carica più registri partendo da indirizzo in R7.
- **Stack:** PUSH {R0-R2}, POP {R0-R2}.
- **Regola d’oro:** Il registro con numero più basso (R0) va sempre all’indirizzo di memoria più basso. L’ordine nelle graffe non conta per l’hardware.

14.5 Stack Framing

- Lo Stack Frame è lo spazio riservato nella RAM per le variabili locali di una funzione.
- **Allocazione:** SUB SP, SP, #X riserva X byte (lo stack cresce verso indirizzi bassi).
 - **Frame Pointer (R7):** MOV R7, SP per avere un riferimento fisso all’inizio del frame.
 - **Accesso:** var. vengono scritte/lette usando offset rispetto a R7, es: STR R0, [R7, #4].
 - **Deallocazione:** Prima del ritorno (BX LR), si deve pulire lo stack con ADD SP, SP, #X.

15 Il Processo di Compilazione

- Il codice sorgente viene trasformato in binario attraverso diverse fasi:
1. **Preprocessore:** Gestisce inclusioni e macro. (#define, #if, ...)
 2. **Compilatore:** Traduce il linguaggio ad alto livello in Assembly.
 3. **Assembler:** Converte Assembly in codice macchina rilocabile.
 4. **Linker/Binder:** Unisce file oggetto e librerie.
 5. **Loader:** Converte indirizzi rilocabili in assoluti e carica in memoria.

15.1 Analisi del Compilatore

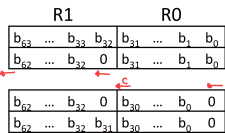
- **Analisi Lessicale:** Rimozione spazi e raggruppamento token.
- **Analisi Sintattica/Semantica:** Controllo struttura e tipi di dati.
- **Ottimizzazione:** Miglioramento del codice per velocità o spazio.
- **Generazione:** Produzione del codice target.

16 Esempi

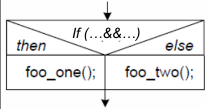
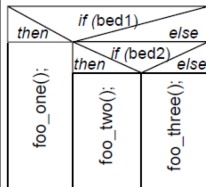
```
1 ;Example: 64Bit Addition
2 LDR r4, =addrOp ;load address of int64 array [op1,op2,res]
3 LDM r4, {r0-r3} ;load op1[r1,r0] and op2[r3,r2]
4 ADDS r0, r0, r2 ;add lower words and set carry
5 ADCS r1, r1, r3 ;add higher words with carry in
6 ADDS r4, #16 ;set r4 to address of res
7 STM r4!, {r0-r1} ;store 64bit result from r0, r1 to res
```

```
1 ; Example: 64Bit Subtraction
2 LDR r4, =addrOp ; load address of int64 array [op1,op2,res]
3 LDM r4, {r0-r3} ; load op1[r1,r0] and op2[r3,r2]
4 SUBS r0, r0, r2 ; sub lower words and set borrow
5 SBCS r1, r1, r3 ; sub higher words with borrow in
6 ADDS r4, #16 ; set r4 to address of res
7 STM r4!, {r0-r1} ; store 64bit result from r0, r1 to res
```

```
1 ;Example for 64bit logical shift left
2 LDR r2, =data64 ;load address of data64
3 LDM r2, {r0-r1} ;load data64 to r0, r1
4 LSLS r1, r1, #1 ;logical shift left r1 by 1
5 MOVS r3, #0 ;r3 = 0
6 LSLS r0, r0, #1 ;lsl r0 by 1 (carry = b31)
7 ADCS r1, r3 ;r1 = r1 + 0 + carry
8 STM r2!, {r0-r1} ;store r0,r1 in data64
```



Equivalente a: (·2)



Thumb-2 Assembly	C Source Code
<pre>LDR R0, =G2 LDR R1, =G1 LDRH R0, [R0] ;R0 ← G2 SXTH R0, R0 LDR R1, [R1] ;R1 ← G1 CMP R0, R1 BGE else_if then: BL foo_one B endif else_if: else_if: CMP R0, R1 BLE else_two BL foo_two B endif else: BL foo_three endif:</pre>	<pre>int32_t G1; // signed 32bit int16_t G2; // signed 16bit ... if(G1 > G2) // G2-G1 < 0? { foo_one(); } else if(G1 > G2) // G2-G1 > 0? { foo_two(); } else // G2 == G1 { foo_three(); }</pre>
<pre>LDR R0, =G1 LDR R0, [R0] ;R0 ← G1 MOVS R1, #4 ;R1 = 4 CMN R0, R1 ;G1-(-4) BLE else BGT else then: BL foo_one B endif else: BL foo_two endif:</pre>	<pre>int32_t G1; // signed 32bit ... if((G1 > -4) && (G1 <= 4)) { foo_one(); } else { foo_thwo(); }</pre>

Thumb-2 Assembly	C Source Code
<pre>0x080000f6 LDR R1, =State 0x080000f8 LDRB R0, [R1] ;R0 ← State 0x080000fa CMP R0, #3 ;Default State R0 - 3 0x080000fc BHI default ;R0 - 3 > 0 0x08000100 LSLS R0, R0, #2 ;R0 = R0 * 4 (offset) * 0x08000102 ADR R1, lookup ;R1 = 0x08000108 0x08000104 LDR R0, [R1, R0] ;R0 = val[lookup + offset] 0x08000106 MOV PC, R0 ** ;PC ← Addr of case lookup: 0x08000108 .word 0x08000130 ;addr of case 0 0x0800010c .word 0x08000136 ;addr of case 1 0x08000110 .word 0x0800013c ;addr of case 2 0x08000114 .word 0x08000142 ;addr of case 3 ... case_0: 0x08000130 BL foo_one 0x08000134 B end_switch case_1: 0x08000136 BL foo_two 0x0800013a B end_switch case_2: 0x0800013c BL foo_three 0x08000140 B end_switch case_3: 0x08000142 BL foo_four 0x08000146 B end_switch default: 0x08000148 MOVS R0, #0 0x0800014a STRB R0, [R1] end_switch: 0x0800014c ...</pre>	<pre>uint8_t State; // unsigned 8bit ... switch(State) { case 0: foo_one(); break; case 1: foo_two(); break; case 2: foo_three(); break; case 3: foo_four(); break; default: State = 0; break; } ...</pre>

1. **Controllo limiti:** Verifica valore di State per saltare subito al default se l’indice è fuori range.
2. **Calcolo offset:** Moltiplicazione del valore per 4 (State << 2) tramite shift logico, poiché ogni puntatore nella look-up table occupa 32 bit (4 byte).
3. **Recupero puntatore:** Lettura dell’indirizzo di destinazione esatto sommando l’offset calcolato all’indirizzo base della tabella.
4. **Branch:** avviare l’esecuzione del caso specifico.

